Session 4: CNN Hands-on Lab

Building and Training CNNs for EO Image Classification

Dimitrios Kasampalis
Anefelo

Session Overview

Duration: 2.5 hours
Type: Intensive hands-on lab
Goal: Turn CNN theory into a working model

You will: - Prepare the EuroSAT dataset - Build a CNN from scratch in TensorFlow/Keras - Train with GPU acceleration in Colab - Evaluate with confusion matrix and F1 - Apply transfer learning (ResNet50)

Prerequisites: - Session 3 complete (CNN basics) - Colab account with GPU enabled - Python & NumPy fundamentals

Notebooks:

`session4_cnn_classification_STUDENT.ipynb`
`session4_transfer_learning_STUDENT.ipynb`

Setup & Data Preparation

Colab GPU + Environment

Steps: 1. Runtime → Change runtime type → Hardware accelerator → GPU
2. Verify with `!nvidia-smi`
3. TensorFlow 2.16+ (Keras 3) is pre-installed on Colab
4. Set seeds for reproducibility

```
import tensorflow as tf, numpy as np, random, os
SEED = 42
tf.keras.utils.set_random_seed(SEED)   # seeds Python, NumPy and TF
print(tf.__version__, tf.config.list_physical_devices('GPU'))
```

EuroSAT Dataset (Sentinel-2, 10 classes)

- ~27k RGB chips (64×64) derived from S2
- Classes: AnnualCrop, Forest, Herbaceous, Highway, Industrial, Pasture, PermanentCrop, Residential, River, SeaLake

```
# Download from the authors' Zenodo record (MIT licence)
url = "https://zenodo.org/records/7711810/files/EuroSAT_RGB.zip"
path = keras.utils.get_file("EuroSAT_RGB.zip", url, extract=True)
# -> one folder per class; shuffle with a fixed seed, split 70/15/15
```

Preprocessing: Normalize to [0,1], integer labels (sparse loss), split 70/15/15

Building a CNN from Scratch

Architecture (reference)

```
Input (64×64×3)
 → [Conv(32, 3×3) + ReLU] → MaxPool
 → [Conv(64, 3×3) + ReLU] → MaxPool
 → [Conv(128,3×3) + ReLU] → MaxPool
 → Flatten → Dropout(0.5)
 → Dense(128) + ReLU → Dropout(0.5)
 → Dense(10) + Softmax
```

```python
from tensorflow.keras import Sequential, Input
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Dropout, Flatten

model = Sequential([
  Input(shape=(64, 64, 3)),
  Conv2D(32, (3,3), activation='relu'),
  MaxPooling2D(),
  Conv2D(64, (3,3), activation='relu'),
  MaxPooling2D(),
  Conv2D(128,(3,3), activation='relu'),
  MaxPooling2D(),
  Flatten(), Dropout(0.5),
  Dense(128, activation='relu'), Dropout(0.5),
  Dense(10, activation='softmax')
])
```

Compile & Train

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])   # integer labels → sparse loss

cb = [
  tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
  tf.keras.callbacks.ModelCheckpoint('best_model.keras', save_best_only=True),  # .h5 is legacy
  tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)
]

norm = lambda x, y: (tf.cast(x, tf.float32) / 255.0, y)
history = model.fit(ds_train.map(norm).shuffle(1000).batch(64).prefetch(2),
                    validation_data=ds_val.map(norm).batch(64).prefetch(2),
                    epochs=30, callbacks=cb)
```

Healthy curves: train↓, val↓ then plateau; small gap
Overfitting: large gap → add dropout/augmentation

Evaluation & Error Analysis

Accuracy + Confusion Matrix

```python
import numpy as np, matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report

y_true, y_pred = [], []
for x, y in ds_test.map(norm).batch(64):
  p = model.predict(x, verbose=0).argmax(axis=1)
  y_pred.extend(p); y_true.extend(y.numpy())

cm = confusion_matrix(y_true, y_pred)
print(classification_report(y_true, y_pred))
```

Look for: - Which pairs are confused? (e.g., AnnualCrop vs Herbaceous)
- Per-class precision/recall balance

Visualize Misclassifications

```
names = meta.features['label'].names
wrong = np.where(np.array(y_true) != np.array(y_pred))[0][:12]
imgs = [img for img, _ in ds_test.as_numpy_iterator()]  # ~4k chips: fits in memory
fig, axes = plt.subplots(3, 4, figsize=(10, 8))
for ax, i in zip(axes.flat, wrong):
    ax.imshow(imgs[i]); ax.set_title(f'T:{names[y_true[i]]}\nP:{names[y_pred[i]]}', fontsize=8); ax.axis('off')
```

- Investigate systematic errors
- Adjust augmentation / architecture accordingly

Data Augmentation

EO-aware augmentations

```python
import tensorflow as tf
from tensorflow.keras import layers

data_aug = tf.keras.Sequential([
  layers.RandomRotation(0.25),
  layers.RandomFlip('horizontal_and_vertical'),
  layers.RandomZoom(0.1),
  layers.RandomContrast(0.1)
])
```

- Rotations/flips OK for overhead imagery
- Brightness/contrast for atmospherics
- Avoid orientation-critical tasks if sensitive (roads)

Transfer Learning (ResNet50)

Feature extraction → fine-tuning

```python
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout, Input
from tensorflow.keras.models import Model

# Feed 0–255 images through preprocess_input (ImageNet BGR mean subtraction), not /255
base = ResNet50(include_top=False, weights='imagenet', input_shape=(64,64,3))
for l in base.layers[:int(0.8*len(base.layers))]:
    l.trainable = False

x = GlobalAveragePooling2D()(base.output)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
out = Dense(10, activation='softmax')(x)
model = Model(base.input, out)
```

Strategy: - Start with frozen base → quick convergence
- Unfreeze top blocks for small accuracy gains

Compare Approaches

Approach	Train time (T4)	Typical accuracy
From scratch	5–10 min	~90–95%
Feature extraction	5–15 min	~93–96%
Partial fine-tune	15–30 min	~95–98%

Indicative — the notebooks measure your own numbers

Tip: Use the fastest path during live sessions, fine-tune offline later

Troubleshooting

Common issues & fixes

- **GPU not detected:** set runtime → GPU; restart runtime
- **OOM error:** lower batch size; fewer filters; mixed precision
- **Accuracy stuck ~10%:** check labels; learning rate; normalization
- **Overfitting:** stronger augmentation; more dropout; L2; early stop
- **Slow training:** reduce model depth; use caching/prefetch

Philippine Context

Why CNNs matter operationally

- National land cover updates
- Cloud masking for S2 mosaics
- Disaster mapping (flood/damage)
- Urban growth monitoring
- Supports DENR, DA, NDRRMC, LGUs

Next steps: move to Day 3 (U-Net segmentation, flood mapping)

Summary & Notebook

What you achieved today

1. Built and trained a CNN classifier (EuroSAT)
2. Evaluated with robust metrics and confusion matrix
3. Applied transfer learning for higher accuracy
4. Learned practical debugging strategies

Notebooks:

`session4_cnn_classification_STUDENT.ipynb`
`session4_transfer_learning_STUDENT.ipynb`